```
    d3.select("svg").node().appendChild(
        d3.select(svgData).select("path").node());
    }
    d3.selectAll("path").attr("transform", "translate(50,50)");
};
```

Notice how we've added a `transform` attribute to offset the paths so that they won't be clipped in the top-right corner. Instead, you clearly see a football in the top corner of your `<svg>` canvas. Document fragments aren't a normal part of your DOM, so you don't have to worry about accidentally selecting the `<svg>` canvas in the document fragment, or any other elements.

A `while` loop like this is sometimes necessary, but typically the best and most efficient method is to use `.each()` with your selection. Remember, `.each()` runs the same code on every element of a selection. In this case, we want to select our `<svg>` canvas and append the path to that canvas.

```
function loadSVG(svgData) {
    d3.select(svgData).selectAll("path").each(function() {
        d3.select("svg").node().appendChild(this);
    });
    d3.selectAll("path").attr("transform", "translate(50,50)");
};
```

We end up with a football floating in the top-left corner of our canvas, as shown in figure 3.22.

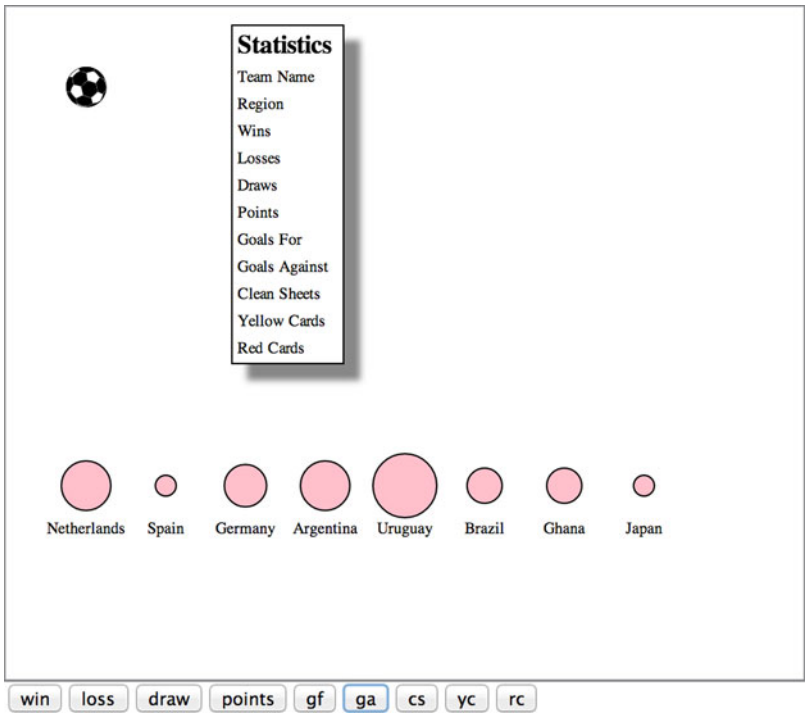


Figure 3.22 A hand-drawn football icon is loaded onto the `<svg>` canvas, along with the other SVG and HTML elements we created in our code.